1. You can create a script file with the file extension .sh

```
kmin@kali-rrc:~$ touch my_first_srcipt.sh
kmin@kali-rrc:~$ ls
bills_keys  bills_keys.pub  Desktop  Documents  Downloads  Music  my_first_srcipt.sh
kmin@kali-rrc:~$ nano my_first_srcipt.sh
kmin@kali-rrc:~$
```

2. Your script has to start with “#!/bin/bash” in the first line to work

```
  GNU nano 5.2
#!/bin/bash

echo hello world
```

3. Change the file permission in order to be executable

```
kmin@kali-rrc:~$ ls -l
total 48
-rw------- 1 kmin kmin  505 Feb 15 12:01 bills_keys
-rw-r--r-- 1 kmin kmin  175 Feb 15 12:01 bills_keys.pub
drwxr-xr-x 2 kmin kmin 4096 Feb 15 11:55 Desktop
drwxr-xr-x 2 kmin kmin 4096 Feb 15 11:34 Documents
drwxr-xr-x 2 kmin kmin 4096 Feb 15 11:34 Downloads
drwxr-xr-x 2 kmin kmin 4096 Feb 15 11:34 Music
-rw-r--r-- 1 kmin kmin   30 Feb 20 11:31 my_first_srcipt.sh
drwxr-xr-x 2 kmin kmin 4096 Feb 15 11:34 Pictures
drwxr-xr-x 2 kmin kmin 4096 Feb 15 11:34 Public
drwxr-xr-x 2 kmin kmin 4096 Feb 15 11:34 Templates
-rw-r--r-- 1 kmin kmin   12 Feb 15 11:44 text
drwxr-xr-x 2 kmin kmin 4096 Feb 15 11:34 Videos
kmin@kali-rrc:~$ chmod 744 my_first_srcipt.sh
```

4. Now, you can run the command by simply writing the file path

```
kmin@kali-rrc:~$ ls -l
total 48
-rw------- 1 kmin kmin  505 Feb 15 12:01 bills_keys
-rw-r--r-- 1 kmin kmin  175 Feb 15 12:01 bills_keys.pub
drwxr-xr-x 2 kmin kmin 4096 Feb 15 11:55 Desktop
drwxr-xr-x 2 kmin kmin 4096 Feb 15 11:34 Documents
drwxr-xr-x 2 kmin kmin 4096 Feb 15 11:34 Downloads
drwxr-xr-x 2 kmin kmin 4096 Feb 15 11:34 Music
-rwxr--r-- 1 kmin kmin   30 Feb 20 11:31 my_first_script.sh
drwxr-xr-x 2 kmin kmin 4096 Feb 15 11:34 Pictures
drwxr-xr-x 2 kmin kmin 4096 Feb 15 11:34 Public
drwxr-xr-x 2 kmin kmin 4096 Feb 15 11:34 Templates
-rw-r--r-- 1 kmin kmin   12 Feb 15 11:44 text
drwxr-xr-x 2 kmin kmin 4096 Feb 15 11:34 Videos
kmin@kali-rrc:~$ ~/my_first_script.sh
hello world
kmin@kali-rrc:~$
```

5. You can also set variables in the script in the script in the format shown below

```
#!/bin/bash

msg='Hello world!'
echo 'The value of msg is currently: $msg'
```

6. You can write input statements making the user write something. Use the “read” command with “-p” option. The variable doesn’t need to start with $ in this case.

```
#!/bin/bash

msg="Hello world!"
echo "The value of msg is currently: $msg"

read -p "Please enter the value of msg2: " msg2
echo "The value of msg2 is: $msg2"
```

variable

7. When you run it, it’ll show up like this.

```
kmin@kali-rrc:~$ ~/my_first_script.sh
Hello world!
kmin@kali-rrc:~$ nano my_first_script.sh
kmin@kali-rrc:~$ ~/my_first_script.sh
The value of msg is currently: $msg
kmin@kali-rrc:~$ nano my_first_script.sh
kmin@kali-rrc:~$ ~/my_first_script.sh
The value of msg is currently: Hello world!
kmin@kali-rrc:~$ nano my_first_script.sh
kmin@kali-rrc:~$ ~/my_first_script.sh
The value of msg is currently: Hello world!
Please enter the value of msg2: 234asdf
The value of msg2 is: 234asdf
kmin@kali-rrc:~$
```

8. You can also save the values in a file using the “echo” command

```
  GNU nano 5.2
#!/bin/bash

msg="Hello world!"
echo "The value of msg is currently: $msg"

read -p "Please enter the value of msg2: " msg2
echo "The value of msg2 is: $msg2"

echo $msg >> output.txt
echo $msg2 >> output.txt
```

9. The result will be as shown.

```
kmin@kali-rrc:~/Desktop$ ~/my_first_script.sh
The value of msg is currently: Hello world!
Please enter the value of msg2: blahblah
The value of msg2 is: blahblah
kmin@kali-rrc:~/Desktop$ ls
output.txt  text2
kmin@kali-rrc:~/Desktop$ cat output.txt
Hello world!
blahblah
kmin@kali-rrc:~/Desktop$
```

10. If you run it again, it’ll create another txt file since we used the relative path. Use the absolute path to avoid that.

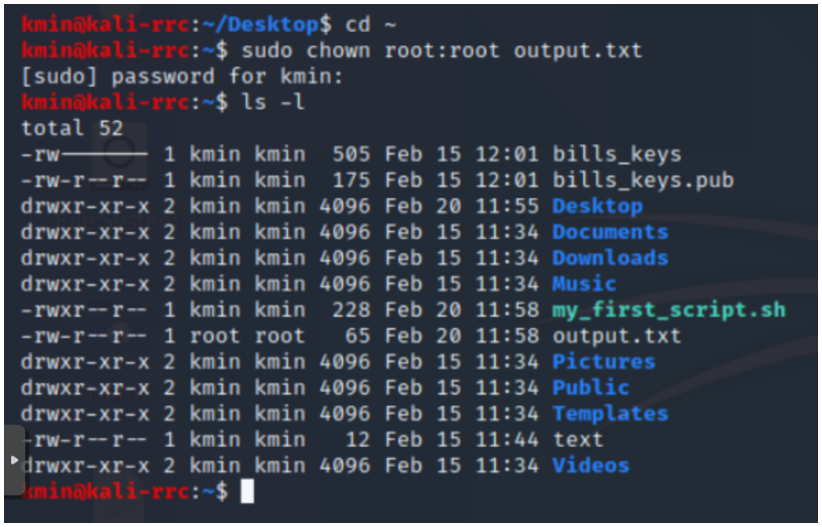
```
#!/bin/bash

msg="Hello world!"
echo "The value of msg is currently: $msg"

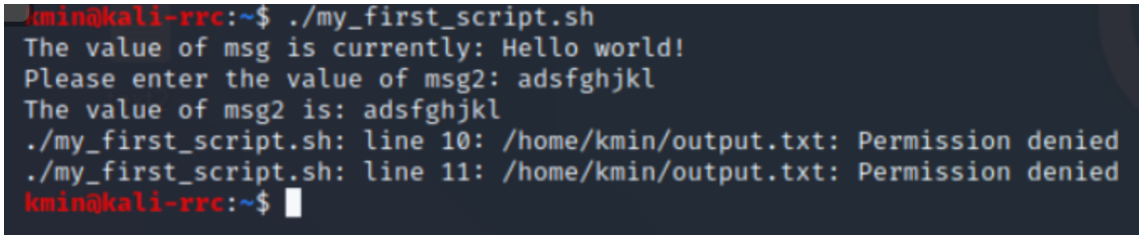
read -p "Please enter the value of msg2: " msg2
echo "The value of msg2 is: $msg2"

path="~/output.txt"
echo $msg >> $path
echo $msg2 >> $path
```

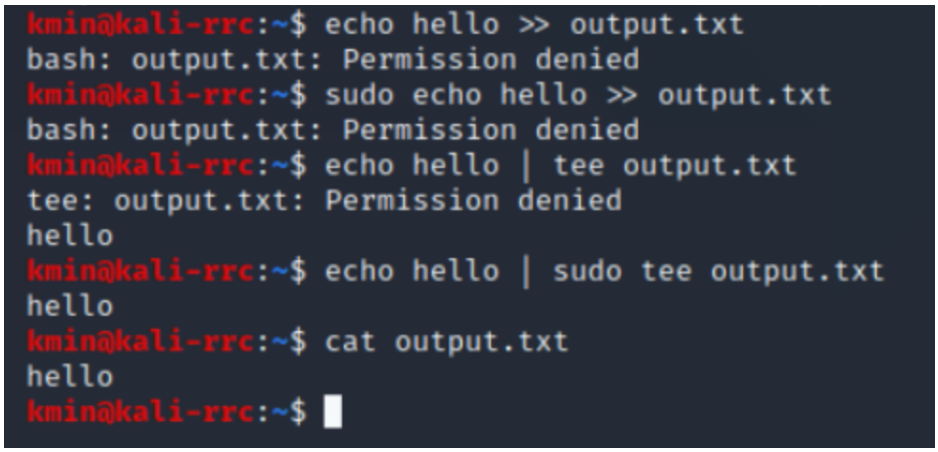
11. Now, let's change the file ownership to root.



12. If you run the command, it won't work.



13. One way to solve the problem is to use | and the "tee" command.



14. The other way is to run the entire script as root

```
kmin@kali-rrc:~$ sudo ./my_first_script.sh
The value of msg is currently: Hello world!
Please enter the value of msg2: i am running as root
The value of msg2 is: i am running as root
kmin@kali-rrc:~$ cat output.txt
hello
Hello world!
i am running as root
kmin@kali-rrc:~$
```

15. This is the script of an ipchanger

```
  GNU nano 5.2                                                    ipchanger.sh
#!/bin/bash

path="/etc/network/interfaces"
echo "# This file describes the network interfaces available on your system" > $path
echo "# and how to activate them. For more information, see interfaces(5)." >> $path
echo "" >> $path
echo "source /etc/network/interfaces.d/*" >> $path
echo "" >> $path
echo "# The loopback network interface" >> $path
echo "auto lo" >> $path
echo "iface lo inet loopback" >> $path
echo "" >> $path
echo "auto eth0" >> $path
echo "iface eth0 inet static" >> $path


read -p "Please enter the new IP address: " ip
read -p "Please enter the new netmask: " mask

echo "  address $ip" >> $path
echo "  netmask $mask" >> $path

systemctl restart networking
ip addr show dev eth0
```

16. You can run the command and see the IP address is changed.

```
kmin@kali-rrc:~$ sudo ~/ipchanger.sh
Please enter the new IP address: 192.168.1.137
Please enter the new netmask: 255.255.255.0
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast state UP group default qlen 1000
    link/ether 00:00:30:f6:3a:9f brd ff:ff:ff:ff:ff:ff
    inet 192.168.1.137/24 brd 192.168.1.255 scope global eth0
       valid_lft forever preferred_lft forever
    inet6 fe80::200:30ff:fef6:3a9f/64 scope link tentative
       valid_lft forever preferred_lft forever
kmin@kali-rrc:~$
```